3970. Shortest Path With At Most K Consecutive Identical Characters

Daily Question - October 30, 2026

Problem Statement

You are given n nodes in a directed weighted graph, numbered 0 to n - 1. Each edges[i] = [ui, vi, wi] represents a directed edge from ui to vi with weight wi. A string labels of length n assigns a lowercase English letter to each node. You are also given an integer k.

Return the minimum total edge weight of a path from node 0 to node n - 1 such that the concatenated node labels contain at most k consecutive identical characters. Include the labels of both endpoints. If no valid path exists, return -1.

Examples

Example 1: n = 3, edges = [[0,1,1],[1,2,1],[0,2,3]], labels = "aab", k = 1. Output: 3. The direct path 0 -> 2 gives "ab" and costs 3. The cheaper path through node 1 gives "aab" and is invalid.

Example 2: n = 3, edges = [[0,1,1],[1,2,1],[0,2,3]], labels = "aab", k = 2. Output: 2. The path 0 -> 1 -> 2 gives "aab", which is valid, and costs 2.

Example 3: n = 3, edges = [[0,1,1],[1,2,1]], labels = "aaa", k = 2. Output: -1. The only route has three consecutive identical characters.

Constraints

1 <= n == labels.length <= 50,000; 0 <= edges.length <= 50,000; edges[i] = [ui, vi, wi]; 0 <= ui, vi < n; ui != vi; 1 <= wi <= 10,000; labels consists of lowercase English letters; 1 <= k <= 50.

Solution: Dijkstra with an Expanded State

Use a state (node, run), where run is the number of consecutive equal labels at the end of the current path. The label itself is determined by the node. Different run lengths at the same node must be tracked separately because they allow different future moves.

Let distance[u][r] be the minimum cost to reach node u with ending run length r. Initialize distance[0][1] = 0 because the starting node contributes one character and no edge cost.

For an edge u -> v, set nextRun = run + 1 if labels[u] equals labels[v], otherwise set nextRun = 1. Ignore the edge if nextRun > k. Otherwise relax distance[v][nextRun] using the edge weight.

Use a minimum-priority queue, discard stale entries, and return when a destination state is removed with its current best distance. Return -1 if the queue empties first.

Complete Java Solution

```
import java.util.Arrays;
import java.util.PriorityQueue;
```

```
class Solution {
    public int shortestPath(
            int n, int[][] edges, String labels, int k) {
        int[] head = new int[n];
        Arrays.fill(head, -1);

        int edgeCount = edges.length;
        int[] to = new int[edgeCount];
        int[] weight = new int[edgeCount];
        int[] next = new int[edgeCount];

        for (int index = 0; index < edgeCount; index++) {
            int u = edges[index][0];
            to[index] = edges[index][1];
            weight[index] = edges[index][2];
            next[index] = head[u];
            head[u] = index;
        }

        char[] characters = labels.toCharArray();
        long[][] distance = new long[n][k + 1];
        for (long[] row : distance) {
            Arrays.fill(row, Long.MAX_VALUE);
        }

        PriorityQueue<State> queue = new PriorityQueue<>(
                (first, second) ->
                        Long.compare(first.cost, second.cost)
        );
        distance[0][1] = 0;
        queue.offer(new State(0, 1, 0L));

        while (!queue.isEmpty()) {
            State current = queue.poll();
            int u = current.node;
            int run = current.run;
            long cost = current.cost;

            if (cost != distance[u][run]) {
                continue;
            }
            if (u == n - 1) {
                return (int) cost;
            }

            for (int edge = head[u]; edge != -1; edge = next[edge]) {
                int v = to[edge];
                int nextRun = characters[u] == characters[v]
                        ? run + 1 : 1;
                if (nextRun > k) {
                    continue;
                }
                long nextCost = cost + weight[edge];
                if (nextCost < distance[v][nextRun]) {
                    distance[v][nextRun] = nextCost;
                    queue.offer(new State(v, nextRun, nextCost));
                }
            }
        }
        return -1;
    }

    private static class State {
        final int node;
        final int run;
        final long cost;

        State(int node, int run, long cost) {
```

```
            this.node = node;
            this.run = run;
            this.cost = cost;
        }
    }
}
```

Correctness

Every valid graph path corresponds to a sequence of expanded states: the run starts at 1 and is updated exactly according to the next label. Conversely, every allowed expanded-state transition corresponds to an original edge and preserves the consecutive-label limit. Thus valid routes and reachable expanded-state walks have identical costs. All edge weights are positive, so Dijkstra settles states in nondecreasing minimum cost. The first settled destination state therefore has the minimum cost among all valid routes.

Complexity

Let E = edges.length. There are at most n*k states and E*k transitions. For this lazy priority-queue implementation, time is O(n*k + E*k*log(E*k + 2)). Space is O(n*k + E + E*k), including possible stale queue entries.

Edge Cases and Implementation Notes

If n = 1, return 0: the start is already the destination and its one-character run is allowed. If no valid route reaches the destination, return -1. Different labels reset the run to 1, not 0. Edges are directed, so no reverse edge is added automatically.

The Java method returns int to match the supplied driver; long is used internally for distance calculations. The final cast assumes the platform expects a representable int result. The supplied statement does not explicitly state a bound on the answer.